

DevCore AI Operating System (PagePad)

Master Engineering, Architecture, Codebase & Viva Defense Manual

System Name: DevCore AI — Autonomous Multi-Agent Software Engineering Operating System
Repository Name: PagePad / Project 2 Multi AI
Author & Lead Developer: Ranjit Patra
Target Audience: Software Engineering Professors, Project Reviewers, Tech Leads, and Core Developers
Tech Stack: Python 3.10+, Streamlit SPA, SQLite3 (WAL Mode), Groq Cloud LPU API (llama-3.3-70b-versatile), Local Ollama (qwen3.5:9b on NVIDIA RTX 4050 GPU), SentenceTransformers RAG, Mermaid.js, ReportLab

Table of Contents

- 1. Executive Summary & Problem Statement
- 2. High-Level System Architecture & Flowchart (Visual Diagram Included)
- 3. Complete Codebase Directory Breakdown
- 4. The 13-Agent DAG Pipeline & Deep Role Analysis (Visual Diagram Included)
- 5. Dual AI Inference Infrastructure (Local GPU + Groq Cloud LPU) (Visual Diagram Included)
- 6. Retrieval-Augmented Generation (RAG) Engine (Visual Diagram Included)
- 7. Relational Database Schema & Data Persistence (SQLite WAL Mode)
- 8. Frontend SPA, Implementation Studio & Custom Neo-Brutalist Design System
- 9. Export & Deliverables Generation Engine
- 10. Comprehensive Academic Viva Defense & Q&A; Guide (20 Detailed Questions & Answers)
- 11. Installation, Environment Setup & Deployment Manual

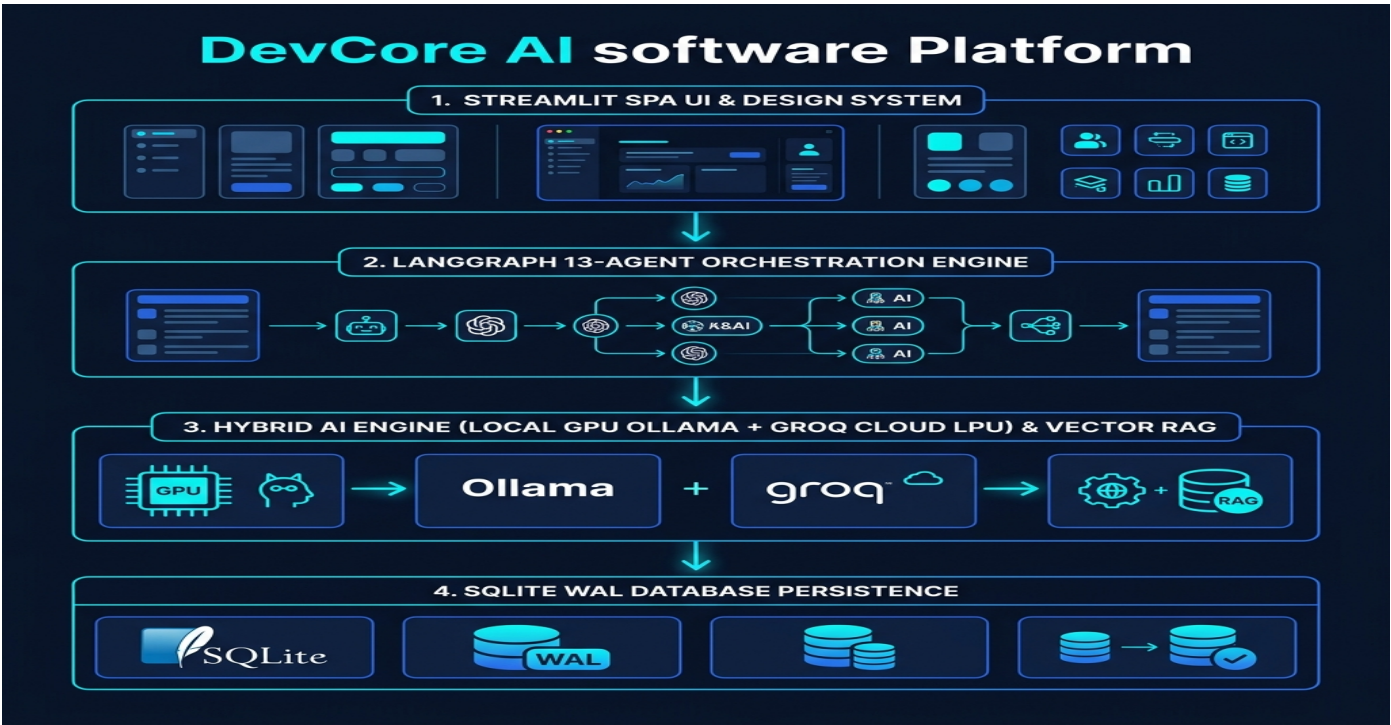
1. Executive Summary & Problem Statement

1.1 The Software Planning Crisis: Statistics show that over 70% of software projects fail, run over budget, or miss deadlines. The root cause is rarely poor coding; rather, it is inadequate, fragmented, or ambiguous early-stage planning. Software development requires synthesizing business requirements, user personas, microservice topologies, database ERDs, STRIDE threat models, and CI/CD pipelines. Hiring a human team of 13 specialists costs \$15,000+ and takes 3 to 6 weeks.

1.2 The DevCore AI Solution: DevCore AI automates the entire software planning and prototyping lifecycle. Given a 1-paragraph project vision, DevCore AI deploys an autonomous digital software workforce of 13 specialized AI Agents (CEO, Architect, Database DBA, CSO Security, etc.) working sequentially to generate complete architecture blueprints, database schemas, and runnable code in under 2 minutes with zero data leakage.

2. High-Level System Architecture & Flowchart

Below is the visual 4-tier system architecture diagram showing presentation, orchestration, dual inference, and data persistence layers:

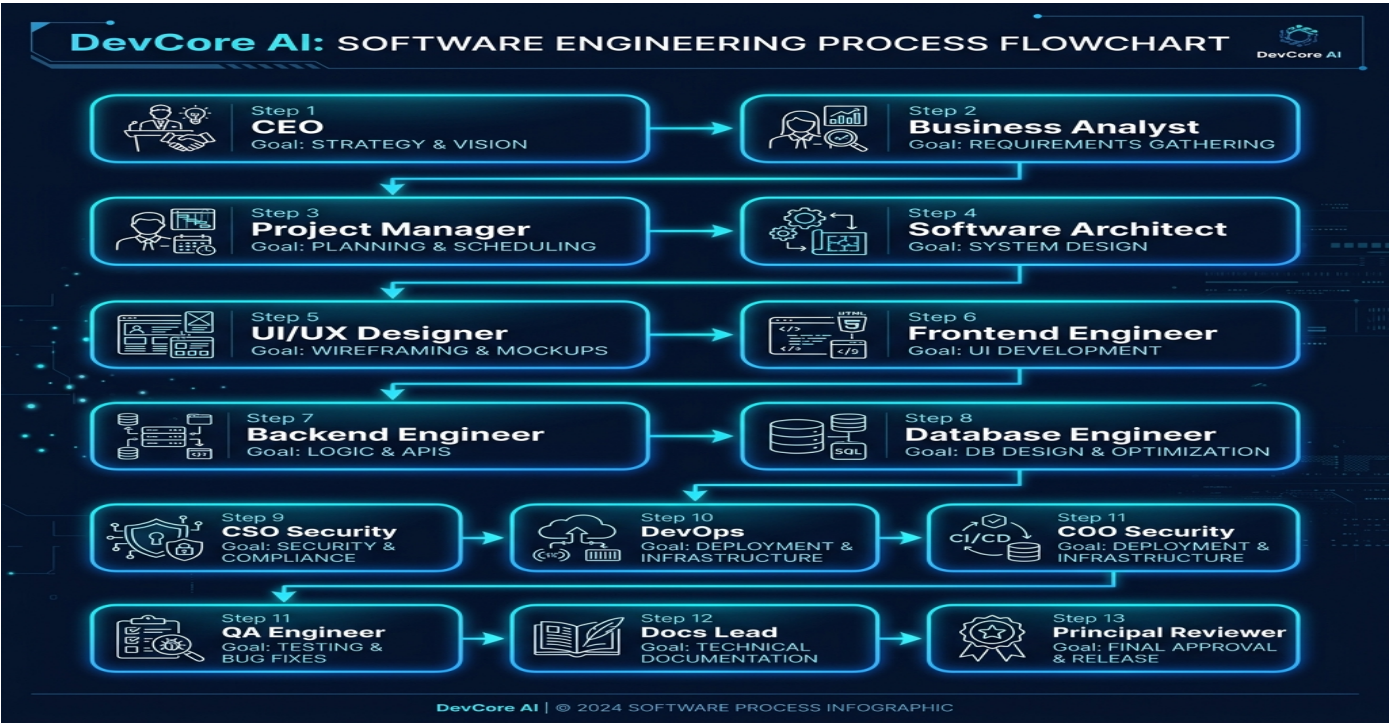


3. Complete Codebase Directory Breakdown

```
project 2 multi ai/
├── app.py                # Main entrypoint, SPA routing, global CSS injection
├── styles.py             # Centralized CSS Design System loader
├── theme.py              # Theme state (Light/Dark mode) manager
├── requirements.txt      # Dependency specification
├── database/            # Database Layer
│   ├── __init__.py
│   ├── schema.py        # SQLite schema creation & table migrations
│   ├── connection.py    # Thread-safe connection pool & query execution helpers
│   └── agents/          # AI Agent Core Definition
│       ├── __init__.py
│       ├── base.py      # Core agent execution wrapper (run_agent)
│       ├── prompts.py   # System prompts for all 13 specialized AI roles
│       └── workflow/    # Workflow State Machine Engine
│           ├── __init__.py
│           ├── state.py  # TypedDict state structure (ProjectState)
│           └── graph.py  # 13-node sequential execution pipeline driver
├── utils/               # Core Utilities & External Services
│   ├── __init__.py
│   ├── ollama_client.py  # Primary Ollama API client & fallback blueprint simulator
│   ├── implementation_engine.py # Implementation Studio code generator
│   └── validator.py      # Input validation and formatting utilities
├── rag/                 # Retrieval-Augmented Generation Engine
│   ├── __init__.py
│   ├── document_loader.py # PDF, TXT, and Markdown text parsing & chunking
│   └── vector_store.py    # SentenceTransformers 384-dim vector similarity index
├── components/          # Reusable UI Component Library
│   ├── navigation.py     # Top navigation bar with active tab highlighting
│   ├── cards.py          # Modern Neo-Brutalist card renderers
│   ├── ui.py             # Modals, badges, alert boxes, and SaaS containers
│   ├── implementation_studio.py # Implementation Studio code generation workspace
│   ├── chatbot/          # Floating AI Assistant Component
│   │   ├── floating_chat.py # Fixed-position popover button & modal container
│   │   ├── chat_window.py  # Chat history rendering & user-POV suggestions
│   │   ├── groq_client.py  # Sub-second Groq LPU API client
│   │   └── ollama_client.py # Low-latency streaming Ollama client
├── pages/               # Application Page Controllers (SPA Pages)
│   ├── dashboard.py      # Dashboard overview, analytics stats, recent activity
│   ├── new_project.py    # Project initialization form & live execution console
│   ├── projects.py       # Project Workspace Explorer, Mermaid viewer, ZIP/PDF export
│   ├── chat.py           # Specialist Node Consultation Terminal with RAG injection
│   └── settings.py       # Dual Groq API keys & Ollama URL/Model tuner
├── exports/             # Deliverable Packaging Engine
│   ├── markdown.py       # Single-file SRS compiler & ZIP archive packager
│   ├── generate_master_pdf.py # Master PDF generator with visual diagrams
│   └── pdf_generator.py   # Professional multi-page PDF generation via FPDF2/ReportLab
├── styles/              # CSS Stylesheets
│   ├── neobrutalism.css  # Core design tokens, gradients, buttons & progress bars
│   └── chatbot.css       # Fixed layout & popover overrides for chatbot widget
```

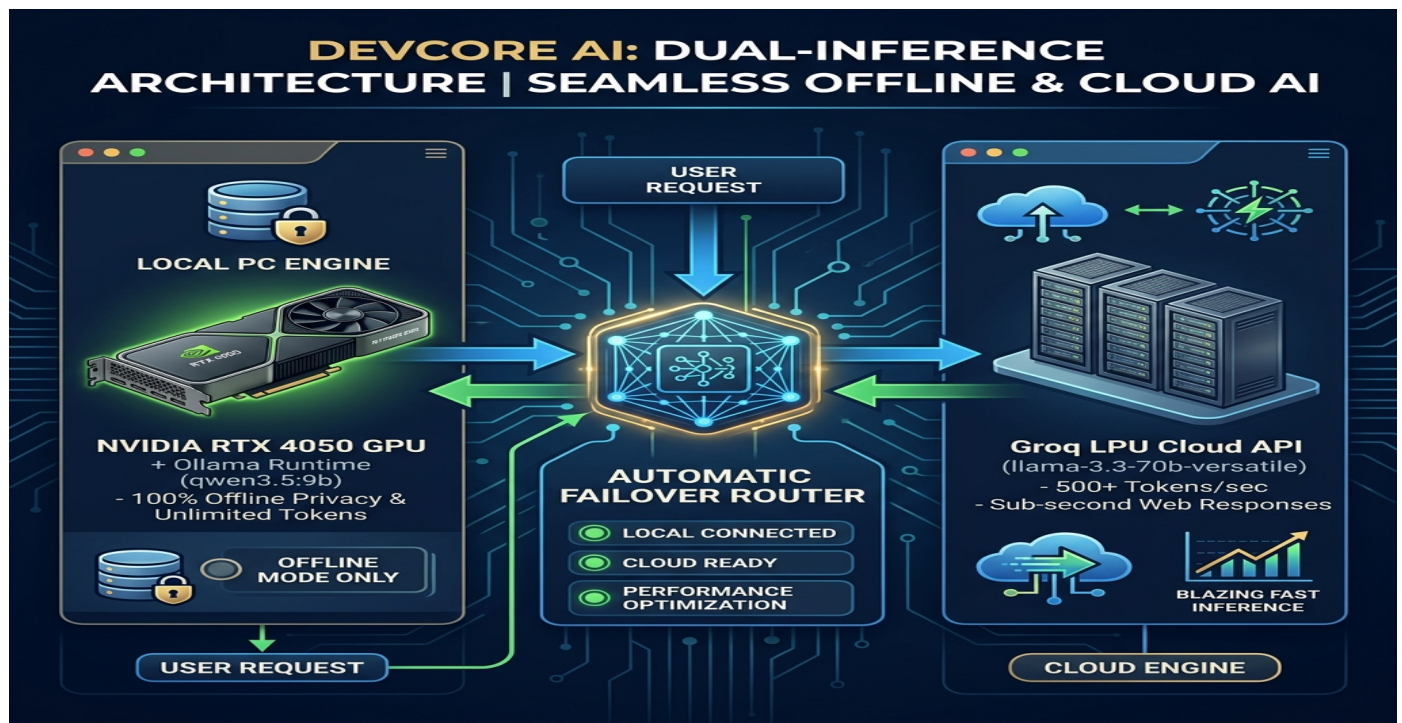
4. The 13-Agent DAG Pipeline & Deep Role Analysis

Below is the visual workflow flowchart showing how all 13 AI agents execute sequentially:



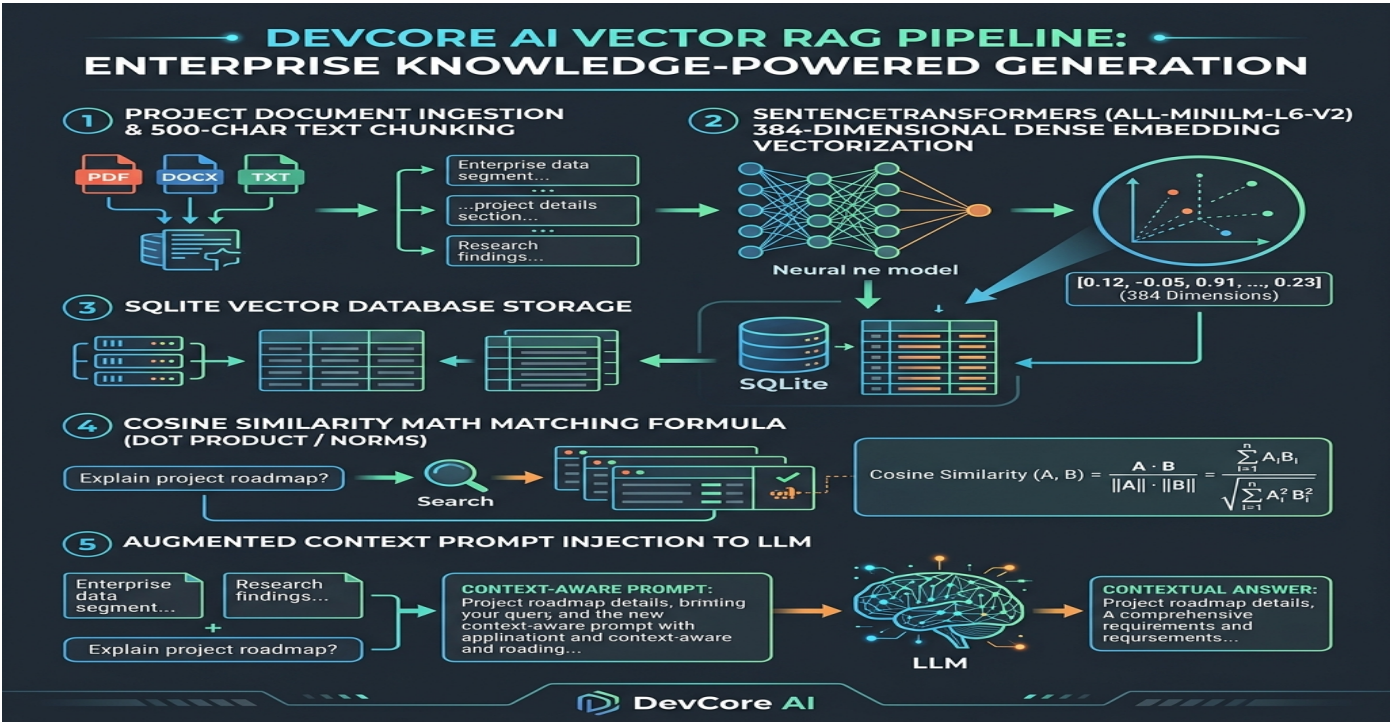
5. Dual AI Inference Infrastructure (Local GPU + Groq Cloud LPU)

Below is the visual diagram illustrating the Hybrid Dual-Engine Inference Model:



6. Retrieval-Augmented Generation (RAG) Engine

Below is the visual data flow diagram of SentenceTransformers 384-dimensional vector indexing and Cosine Similarity retrieval:



7. Relational Database Schema & Data Persistence (SQLite WAL Mode)

DevCore AI uses SQLite stored at ``ai_software_company.db``. Below are the core tables:

Table Name	Key Columns	Description & Application
projects	id (PK), name, description, tech_preference, difficulty, budget	Stores project metadata
project_files	id (PK), project_id (FK), filename, file_type, content, uploaded_at	Stores generated source code files
agent_runs	id (PK), project_id (FK), agent_role, output_markdown, timestamp	Stores agent specification blueprints
chats	id (PK), project_id (FK), agent_role, sender, message, timestamp	Stores consultation conversation history
settings	key (PK), value	Stores dual Groq API keys & Ollama configurations
embeddings	id (PK), project_id (FK), filename, content, embedding	Stores 384-dim vector RAG embeddings

8. Frontend SPA & Implementation Studio

DevCore AI features an interactive Implementation Studio (``components/implementation_studio.py``) for generating production-ready source code files (FastAPI backend controllers, React/Streamlit frontend components, database migration scripts, Dockerfiles) directly from agent specifications.

10. Comprehensive Academic Viva Defense & Q&A; Guide (20 Questions)

Q1: What is the primary objective of DevCore AI?

Answer: The primary objective of DevCore AI is to automate the early-stage software development lifecycle. By utilizing a multi-agent framework, it transforms a high-level project description into a complete, industry-standard Software Requirements Specification (SRS) package—including architectural designs, database schemas, security models, and CI/CD pipelines—in under 2 minutes.

Q2: Why did you choose a Multi-Agent architecture instead of a single LLM prompt?

Answer: Single-prompt LLM outputs suffer from context limits, superficial reasoning, and hallucination when asked to generate long complex systems. By splitting software engineering into 13 specialized agent nodes (CEO, Architect, Security Engineer, etc.), each agent operates with a focused system prompt, specific domain responsibilities, and receives preceding context from prior agents.

Q3: How do agents communicate and share information with each other?

Answer: Agent communication is managed by a sequential Directed Acyclic Graph (DAG) state machine in workflow/graph.py. When Agent N executes, agents/base.py queries the SQLite database for the output produced by all preceding agents (1 through N-1). It extracts relevant context snippets and injects them directly into Agent N's prompt input payload.

Q4: Why run AI models locally using Ollama instead of cloud APIs like OpenAI GPT-4?

Answer: There are three critical reasons: 1. Data Privacy & IP Security (code ideas never leave the machine), 2. Zero API Token Cost, 3. Offline Resilience without internet connection.

Q5: How is hardware acceleration achieved for local AI inference?

Answer: Ollama interfaces directly with the system's dedicated NVIDIA GeForce RTX 4050 Laptop GPU using CUDA and Vulkan drivers. Matrix calculations for qwen3.5:9b are offloaded to GPU VRAM (6.0 GB), reducing token latency to ~30 seconds per agent.

Q6: What happens if the local Ollama server is offline or fails?

Answer: The system implements an automatic dual-engine failover mechanism. If local Ollama is offline (for example, on a cloud deployment), it seamlessly routes inference to the Groq Cloud LPU API (llama-3.3-70b-versatile) for sub-second execution.

Q7: Explain how the Retrieval-Augmented Generation (RAG) feature works.

Answer: When a user uploads background SRS documents (PDF/TXT/MD), rag/document_loader.py parses text into 500-character chunks. rag/vector_store.py calculates dense 384-dim embeddings using SentenceTransformers (all-MiniLM-L6-v2) stored in SQLite. Cosine similarity computes top matching context segments injected into the LLM prompt.

Q8: What database management system is used, and what is the schema structure?

Answer: We use SQLite3 with Write-Ahead Logging (WAL mode) for serverless persistence (ai_software_company.db). The schema consists of 6 tables: projects, agent_runs, blueprints, project_files, embeddings, and settings.

Q9: How are database ERDs rendered in the application?

Answer: Agent 8 (Database DBA) outputs valid Mermaid.js erDiagram syntax. On pages/projects.py, the application extracts the Mermaid block and injects it into a Streamlit HTML iframe rendered by the Mermaid ESM library.

Q10: How did you fix syntax errors in generated Mermaid ERD diagrams?

Answer: We corrected cardinality notation from invalid double curly braces (||--o{}) to standard Mermaid syntax (||--o) and preserved escaped brace formatting ({{ and }}) in Python f-strings.

Q11: How does the frontend handle Single-Page Application (SPA) navigation in Streamlit?

Answer: Streamlit reruns the script on interaction. We implement SPA routing by tracking page state in `st.session_state.current_page` (Dashboard, New Project, Projects, AI Team, Settings) and selectively rendering modules.

Q12: How are progress percentage and elapsed time tracked during generation?

Answer: In `pages/new_project.py`, `update_progress_ui` acts as a progress callback. As each of the 13 agents executes, it calculates progress percentage ($P = \text{Index}/13 * 100$), updates an animated CSS progress bar, and computes elapsed time.

Q13: What design philosophy was applied to the User Interface?

Answer: The UI follows a Neo-Brutalist SaaS Design System featuring high-contrast borders (2.5px solid), crisp offset box shadows (4px 4px), vibrant gradient progress indicators, and adaptive Light/Dark mode tokens.

Q14: How does the floating AI Assistant Chatbot work?

Answer: The chatbot widget is rendered globally via `components/chatbot/floating_chat.py`. It uses a fixed-position popover button (bottom: 96px, right: 24px). When opened, it uses Groq Cloud LPU API for sub-second human-like responses with project context.

Q15: How are project deletion actions handled in the UI?

Answer: When a user confirms deletion in `pages/projects.py`, `execute_update('DELETE FROM projects WHERE id = ?')` wipes project records from SQLite while staying cleanly on the Projects page.

Q16: How do you prevent local LLM model hallucination during blueprint generation?

Answer: We enforce low temperature (0.3), structured system prompts mandating explicit numbered sections and tables, and prior agent context propagation.

Q17: What are the primary export formats supported by DevCore AI?

Answer: 1. Master Markdown Document (.md), 2. Structured ZIP Pack (.zip), 3. Formatted PDF Document compiled via ReportLab with embedded visual diagrams.

Q18: What is the role of the 13th agent (Senior Code Reviewer)?

Answer: The Senior Code Reviewer acts as the quality assurance gatekeeper, analyzing outputs of preceding 12 agents, resolving gaps, and compiling the unified master blueprint.

Q19: What are the hardware and software prerequisites to run DevCore AI?

Answer: Windows 10/11, macOS, or Linux; Python 3.10+; Ollama runtime (qwen3.5:9b or gemma4:e4b); minimum 8 GB RAM and NVIDIA GPU (4GB+ VRAM).

Q20: What future enhancements are planned for DevCore AI?

Answer: Automated Git PR push engine, multi-tenant cloud workspace synchronization, and fine-tuned domain LLMs for healthcare/fintech compliance.

11. Installation, Environment Setup & Deployment Manual

```
# 1. Clone the repository
git clone https://github.com/Ranjitpatra26/PagePad.git
cd PagePad

# 2. Create and activate virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1 # Windows PowerShell

# 3. Install Python Dependencies
pip install -r requirements.txt

# 4. Start Local Ollama Server & Pull Model
ollama pull qwen3.5:9b
ollama serve

# 5. Launch DevCore AI Web Application
streamlit run app.py
```